

# 可行性分析

---

## 1 技术可行性分析 (Technical Feasibility)

本项目的核心逻辑链条为：“数据采集→风格识别（聚类）→策略匹配→仿真验证→OTA 推送”。现有技术栈完全能够支撑这一流程。

### 1.1 数据采集环节

需求：获取加速踏板、车速、制动踏板、方向盘转角等时序数据。

- 方案 A (工业级 / 高精度): Vector CANoe + CAN 卡

适用性：适合车企内部研发，精度高，支持 DBC 文件解析，但成本极高。

- 方案 B (高性价比 / 科研首选): SavvyCAN + CANable (或 PCAN)

适用性：SavvyCAN 是开源工具，配合廉价的 USB-CAN 硬件，可以完美实现 CAN 总线数据的抓取、过滤和图形化显示。

- 方案 C (极低成本 / 演示用): 手机 GPS Logger + ELM327 OBD 蓝牙模块

适用性：成本极低。通过手机 Python 脚本调用 OBD 接口读取标准 PID 数据 (车速、RPM、油门开度)。

结论：硬件接口 (OBD-II/CAN) 是汽车标配，数据获取无技术壁垒。

同时，可以通过手机上的传感器获取车辆的加速度，GPS 位置数据等等，通过以上的数据获得的是实际的数据；还可以通过公开的数据集：

公开数据集：Udacity、NGSIM、China City Driving Cycle 等驾驶行为数据集

示例：Kaggle 上的驾驶数据集：

<https://www.kaggle.com/datasets/outofskills/driving-behavior/data>

Data Card 数据卡Code (48) 代码 (48)Discussion (3) 讨论 (3)Suggestions (0) 建议 (0)

Data recorded on the same portion of road for all the driving styles (NORMAL, SLOW, AGGRESSIVE). It can be treated as a time series.

| AccY                           | # AccZ                  | # GyroX                     | # GyroY                     | # GyroZ                     | A Class                      |
|--------------------------------|-------------------------|-----------------------------|-----------------------------|-----------------------------|------------------------------|
| Acceleration on Y axis. Y 加速度。 | Acceleration on Z axis. | Rotation on X axis. X 轴 旋转。 | Rotation on Y axis. Y 轴 旋转。 | Rotation on Z axis. Z 轴 旋转。 | Driving behavior class. 驾驶行为 |
|                                |                         |                             |                             |                             | SLOW 慢速                      |
| 0.62                           | -6.11                   | -0.51                       | -1.31                       | -0.98                       | NORMAL 正常                    |
| 4.31                           | 5.56                    | 1.49                        | 1.71                        | 0.72                        | Other (814) 其他               |
| 0.2177906                      | 0.45726347              | 0.0                         | 0.0                         | 0.0                         | AGGRESSIVE                   |
| 0.03861046                     | 0.23141575              | -0.054367006                | -0.0077121737               | 0.22525656                  | AGGRESSIVE                   |
| 7.5841208                      | 2.390926                | 0.023823744                 | 0.01366811                  | -0.038026363                | AGGRESSIVE                   |
| 7.6217537                      | 2.5290236               | 0.05681047                  | -0.18058704                 | -0.05207626                 | AGGRESSIVE                   |
| 3.7556214                      | 2.22464                 | -0.03176499                 | -0.03520111                 | 0.035277467                 | AGGRESSIVE                   |
| 0.6468334                      | 0.06529236              | -0.01038471                 | -0.024205536                | 0.0010690142                | AGGRESSIVE                   |
| 0.029413342                    | 0.33282852              | 0.0012217305                | -0.015042556                | 0.023060163                 | AGGRESSIVE                   |
| 22891134                       | 0.35466862              | 0.070249505                 | 0.050320026                 | 0.07742717                  | AGGRESSIVE                   |
| 0.027441323                    | -0.039779663            | 0.040927973                 | -0.010155635                | 0.1366811                   | AGGRESSIVE                   |
| 0.07373035                     | 0.0030345917            | 0.0036651916                | -0.0077121737               | 0.2576324                   | AGGRESSIVE                   |

## 1.2 数据处理与算法模型

需求：特征工程与驾驶风格聚类。

工具推荐：Python 生态

- 数据清洗 (Pandas): 用于处理时间序列数据，补全缺失值，去除噪声 (如  $3\sigma$  法则)。
- 特征提取 (NumPy): 计算加速度变化率、踏板波动率等指标。
- 聚类算法 (Scikit-learn): 直接调用 K-Means 或 GaussianMixture 模型，实现将驾驶员分为“激进”、“温和”等类别。

结论：Python 是处理此类任务的行业标准，算法成熟且有大量开源代码可参考。

步骤：

1. 读取数据：需要的数据有车辆的运动特征（六个自由度的加速度等等）
  1. 数据的处理（可以进行特征的降维等等）
2. 特征提取：

1. 特征标准化标准化
2. 选择聚类的数目（也就是我们要分类的数目）
3. 将聚类之后的驾驶行为打上标签

## 1.3 车辆建模与策略验证

需求：搭建车辆动力学模型，验证不同策略下的能耗差异。

- 方案 A (专业仿真):MATLAB / Simulink

优势：行业权威。可使用内置整车模型，修改参数模拟不同风格并观测 SOC 变化。

- 方案 B (轻量化编程):Python

优势：灵活免费。通过物理公式计算能耗，适合快速验证核心假设。

## 2 经济与操作可行性 (Economic & Operational)

- 硬件依赖低：方案核心在于软件挖掘潜力，不依赖昂贵硬件，利用现有车联网模块进行 OTA，边际成本接近于零。
- 用户接受度优化：针对可能产生的“体感不适”，方案提出了“渐进式适配”机制（如每天调整 30%），具备高落地性。

## 3 推荐使用的核心工具组合 (The "Golden Stack")

建议采用以下高性价比组合以完成可行性演示或论文：

| 环节      | 推荐工具                  | 推荐理由               |
|---------|-----------------------|--------------------|
| 1. 数据采集 | ELM327 (硬件)           | 成本最低，易于获取真实数据。     |
|         | Python-OBD            |                    |
| 2. 数据清洗 | Python (Pandas)       | 强大的时间序列处理能力。       |
| 3. 风格识别 | Python (Scikit-learn) | 几行代码实现 K-Means 聚类。 |
| 4. 节能仿真 | MATLAB / Simulink     | 权威性强，方便生成能耗对比曲线。   |

## 策略

### 可以选择的策略

| 策略维度    | 激进型驾驶风格 (高能耗)                | 温和型/经济型驾驶风格 (目标：极致优化)            | 谨慎/预测型驾驶风格 (目标：平滑舒适)        |
|---------|------------------------------|----------------------------------|-----------------------------|
| 扭矩与功率管理 | 限制瞬间峰值功率，平滑加速曲线，减少无效急加速。     | 优化电机效率区间，使动力输出始终偏向电机高效区。         | 提前计算平顺加速曲线，追求“丝滑”而非迅速的加速感。  |
| 动能回收策略  | 采用高强度回收，最大化利用减速能量，贴近“单踏板”模式。 | 采用标准或自适应回收，根据车速、路况微调，平衡效率与舒适。    | 采用低强度或预测性回收，提前平滑介入，避免突兀拖拽感。 |
| 热管理策略   | 预测性加强电池/电机冷却，避免因大负荷导致的高温降效。  | 允许电池/电机在更宽的高效区间运行，减少温控系统能耗。      | 基于导航和驾驶习惯，预先温和调整电池温度至最优范围。  |
| 附件系统管理  | 智能调节空调功率，或在急加速时短暂降低非关键附件功耗。  | 严格管理附件功耗（如优化空调压缩机频率、降低娱乐系统峰值功率）。 | 附件系统无特殊限制，以保证乘坐舒适性为优先。      |
| 高级集成策略  | 结合导航，在已知弯道、下坡前预调整回收与扭矩图。     | 结合导航与实时路况，规划全局最优能耗路径与速度建议。       | 基于导航，提前更长距离、更柔和地调整车辆状态。     |

| 策略维度    | 激进型驾驶风格（高能耗）                             | 温和型/经济型驾驶风格（目标：极致优化）                   | 谨慎/预测型驾驶风格（目标：平滑舒适）                     |
|---------|------------------------------------------|----------------------------------------|-----------------------------------------|
| 学习与适应机制 | 系统持续学习驾驶者的激进边界，在 <b>安全阈值</b> 内提供最大化性能体验。 | 系统学习驾驶者对经济性的偏好程度， <b>不断微调策略</b> 以贴合习惯。 | 系统学习驾驶者对平顺性的敏感度， <b>个性化调整</b> 所有控制的柔和度。 |

## 如何得到相应的策略

也就是实现方式

1. 高层（也就是基于规则的）：根据驾驶风格分类，选择当前适用的“策略模式”（如激进节能模式、经济巡航模式）
2. **基于优化算法**：对每个类别，将策略参数作为优化变量，以百公里电耗为优化目标，使用遗传算法（GA）、粒子群算法（PSO）等进行自动寻优。
3. 在该模式下，由强化学习智能体实时决策具体的扭矩分配、回收强度等控制指令，以达成高层模式设定的能效与体验目标。（基于强化学习）

## 仿真验证（使用 SUMO 交通仿真）

下载地址：<https://www.eclipse.org/sumo/>

Sumo 是一个开源的、微观的、多模态的交通仿真软件，可以从 OpenStreetMap 导入真实地图。其中每一辆车都是独立的个体，有自己的速度、加速度、行驶路线，会自主跟车、换道、避让。能模拟真实的驾驶行为（急刹车、超车等）也可以向驾驶员发布信息。可以输出：车辆轨迹（位置、速度、加速度）、交通流量统计。SUMO 内置排放模型，可以计算排放数据（CO<sub>2</sub>、NO<sub>x</sub> 等）、能耗数据、交通流量统计。

### 阶段一：

1. 在Simulink中搭建模型：
  - **基础模型**：建立一个包含电池、电机、主减速器的**纯电动车纵向动力学模型**。输入是需求扭矩，输出是车速、加速度、电池电流/电压等。
  - **关键接口**：创建两个代表“驾驶风格”的**输入端口**。例如，一个连续信号（0代表温和，1代表激进），或一个触发信号。同时，模型需能输出**电池瞬时功率**或**SOC变化率**作为能耗计算依据。

- **运行工况**：你需要一个**驾驶员模型**（如PID速度跟随器）来让车辆跟踪一个**标准驾驶循环**（如WLTC、UDDS），或你采集的真实速度曲线。

## 2. 使用Python强化学习进行训练：

- **核心挑战**：让Python RL智能体与Simulink模型实时交互。
- **最佳解决方案**：使用 **MATLAB Engine API for Python**.
  - 在Python脚本中启动MATLAB引擎，然后通过它**直接运行和控制Simulink模型**，就像调用一个函数。
  - **流程**：Python智能体生成一个“动作”（如扭矩指令和风格标签）→ 通过引擎设置到Simulink模型 → 运行模型一个仿真步长 → 获取“状态”（如车速、SOC）和“奖励”（基于能耗计算）→ 智能体学习更新。
- **奖励函数设计**：这是训练成功的关键。示例： $\text{奖励} = - (\text{瞬时功率} * \text{时间权重}) - (\text{速度跟踪误差惩罚}) - (\text{加加速度变化率惩罚})$ 。对于“激进”风格，可以降低对加加速度变化率的惩罚，允许更快的响应。

## 阶段二：策略封装

- 训练完成后，你会得到一个训练好的神经网络（`.pth` 或 `.h5` 文件）。
- 你需要将此网络**封装成一个独立的控制器函数**。这个函数以 **当前车辆状态** 和 **驾驶风格标签** 为输入，**直接输出最优的扭矩指令**，而无需再进行耗时的RL在线学习。

## 阶段三：SUMO 交通流验证

这是验证策略普适性和实际效果的关键。

### 1. 在SUMO中创建场景：

- 构建或导入一个**真实道路网络**（如某个城市街区）。
- 使用 `duarouter` 生成具有真实交通流的**车辆路径**，模拟拥堵、畅通等路况。

### 2. 将策略接入SUMO：

- SUMO通过 **TraCI (Traffic Control Interface)** 接口实现外部控制。
- **核心思路**：编写一个**Python控制脚本**，通过TraCI实时获取某辆受控车的状态（瞬时速度、位置、与前车距离）。
- 将这些状态和预设的“驾驶风格”输入你**封装好的策略控制器**，计算出下一时刻的**目标加速度或速度**。
- 通过TraCI的 `setSpeed` 或 `slowDown` 等命令，将这个目标施加给SUMO中的车辆。

### 3. 对比实验设计：

- **实验组**：一辆车使用个性化RL策略。
- **对照组**：一辆车使用SUMO默认的跟驰模型（如 `IDM`）。
- **关键指标**：在相同的路线和交通流背景下，统计两辆车的**总能耗**（SUMO的 `getElectricityConsumption` 函数可直接获取，或根据模型计算）。

## 4 参考文献与学习资料指引

### 4.1 关键词 (Keywords)

- Driving Style Recognition (驾驶风格识别)
- K-Means Clustering Driver Behavior (K-Means 驾驶行为聚类)
- EV Energy Management Strategy (电动汽车能量管理策略)

### 4.2 核心参考文献

1. 王金鹏。基于风格识别与工况预测的智能车节能速度优化 [D]. 吉林大学, 2023.



表 2.3 特征指标样本

| 车辆参数              | 特征参数                                                                  |
|-------------------|-----------------------------------------------------------------------|
| 纵向加速度             | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 横向加速度             | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 纵向速度              | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 横向速度              | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 纵向加速度变化率<br>(冲击度) | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 横摆角速度             | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 加速踏板位置            | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |
| 制动力               | Max、Min、Mean、SD、RMS、C·V、Me、R、Max-Mean、Mean-Min、Max-Me、Q1、Q3、Mean(>Q3) |

## 5 结论 (Conclusion)

该项目在技术上拥有成熟的工具链，在经济上具有软件定义的低成本优势，且符合节能减排的行业趋势。建议通过小规模数据采集 (OBD 盒子 + 私家车) 跑通“采集 - 分析 - 聚类”流程，作为可行性验证的坚实证据。